From ML Basics to Deep Learning

Review, Linear Regression Math, and Introduction to Neural Networks

1

Part 1: ML Review - What We Covered

2

The 4 Ingredients of Machine Learning

- **1. Data**: The raw material the machine learns from.
- **2. Model**: The mathematical representation (e.g., neural network, decision tree).
- **3. Objective Function (Loss)**: Tells the model how well it's doing. Goal: minimize this.
- **4. Learning Algorithm**: The method to train the model (e.g., Gradient Descent).

3

ML Paradigms Recap

- **Supervised Learning**: Learn from labeled input-output pairs. Tasks: Classification, Regression.
- **Unsupervised Learning**: Find hidden patterns in unlabeled data. Task: Clustering.
- **Reinforcement Learning**: Learn through interaction with an environment via rewards.

4

Techniques We Covered

- **Linear Regression**: Fit a line to predict continuous values. Minimizes squared error.
- **k-Nearest Neighbors (kNN)**: Classify by majority vote of k nearest neighbors.
- **K-Means Clustering**: Partition data into K clusters by iteratively updating centroids.

5

Part 2: ML Review - Topics We Didn't Cover

6

The Labeled Data Bottleneck

- Labeled data is expensive and time-consuming to create.
- The world is full of **unlabeled data** (Wikipedia, YouTube, web pages).
- How can we leverage this massive resource?

7

Semi-Supervised Learning

- Use a **small amount** of labeled data + a **large amount** of unlabeled data.
- The model learns structure from unlabeled data to improve decisions.
- **Example**: Label 100 images, use 10,000 unlabeled images to learn general features.

8

Transfer Learning

- Take a model **pre-trained** on a massive dataset and adapt it for a specific task.
- **Supervised Fine-Tuning (SFT)**: Adapt using your smaller labeled dataset.
- Why it works: Low-level features (edges, textures) are universal across tasks.

9

Self-Supervised Learning (The Key to LLMs)

- Create a supervised task **from the unlabeled data itself**.
- **Example**: Hide a word in a sentence, predict it. The data IS the label!
- This is how GPT, Gemini, and LLaMA learn language by predicting the next word.

10

Reinforcement Learning

- **Agent**: The learner (robot, game AI).
- **Environment**: The world it interacts with.
- **Action**: Moves the agent can make.
- **Reward**: Feedback (positive or negative).
- **Policy**: Strategy for choosing actions to maximize total reward.

11

RL Applications

- **Game Playing**: AlphaGo, AlphaStar, OpenAI Five
- **Robotics**: Learning to walk, grasp objects
- **Resource Optimization**: Data center cooling (Google DeepMind)
- **LLM Fine-tuning**: RLHF (Reinforcement Learning from Human Feedback)

12

Part 3: Linear Regression - The Math Behind It

13

Linear Regression: Problem Setup

- Given m data points with n features and t targets:
- **Predictors**: Matrix X of shape $(m \times n)$
- **Targets**: Matrix Y of shape $(m \times t)$
- **Goal**: Find W and b such that $Y \approx XW + b$

🚀 **Interactive Demo:** ../L02/linear_regression_demo.html

14

The Optimization Problem

- We want to minimize the **Mean Squared Error (MSE)**:
- $$W^*, b^* = \underset{W,b}{\operatorname{argmin}} ||Y - (XW + b)||^2$$
- Here W is $(n \times t)$ and b is length t.

15

The Bias Absorption Trick

- Let $X_{extend} = [X, \mathbf{1}]$ — append a column of 1s to X
- Let $W_{extend} = \begin{bmatrix} W \\ b \end{bmatrix}$
- Now: $Y \approx X_{extend} W_{extend}$
- This simplifies the math significantly!

16

The Closed-Form Solution

- Taking the derivative and setting to zero gives us:
- $$W^*_{extend} = (X^T_{extend} X_{extend})^{-1} X^T_{extend} Y$$
- This is the **Normal Equation** — a direct solution!
- No iteration needed. One computation gives the optimal answer.

17

When Closed-Form Fails

- **Computational Cost**: Matrix inversion is $O(n^3)$ — infeasible for millions of features.
- **Numerical Stability**: $X^T X$ may be singular or near-singular.
- **Non-linear Models**: Neural networks have no closed-form solution.
- **Solution**: Use iterative methods like Gradient Descent.

18

Part 4: From ML to Deep Learning

19

The AI → ML → DL Hierarchy

- **Artificial Intelligence (AI)**: Any system that exhibits intelligent behavior
- **Machine Learning (ML)**: Subset of AI that learns from data (not just rules)
- **Deep Learning (DL)**: Subset of ML using neural networks with many layers
- We've covered AI and ML basics. Now we dive into **Deep Learning**!

20

What is Deep Learning?

- Neural networks with **many layers** (hence 'deep')
- Each layer learns increasingly **abstract features**:
- Layer 1: edges, colors → Layer 2: shapes → Layer 3: objects → ...
- **Key insight**: The network learns its own features — no manual engineering!
- This is what makes DL so powerful for images, text, speech, etc.

21

Why Deep Learning Now?

- **Big Data**: Internet generated massive training datasets
- **GPU Computing**: Parallel processing made training feasible
- **Algorithmic Advances**: Better architectures, optimizers, techniques
- **Breakthroughs**: ImageNet (2012), AlphaGo (2016), GPT (2018-now)
- DL now powers: image recognition, language models, self-driving cars, and more

22

Part 5: From Neurons to Neural Networks

23

Biological Inspiration

- The brain has ~86 billion neurons connected by ~100 trillion synapses.
- **Dendrites**: Receive signals from other neurons.
- **Cell Body (Soma)**: Processes incoming signals.
- **Axon**: Transmits output signal to other neurons.
- **Synapse**: Connection point with adjustable strength.

🚀 **Interactive Demo:** biological_neuron_demo.html

24

The Artificial Neuron (Perceptron)

- **Inputs**: $x_1, x_2, \ldots, x_n$ (like signals from dendrites)
- **Weights**: $w_1, w_2, \ldots, w_n$ (like synaptic strengths)
- **Weighted Sum**: $z = \sum_{i=1}^{n} w_i x_i + b$
- **Activation**: $a = f(z)$ where f is an activation function

🚀 **Interactive Demo:** perceptron_demo.html

25

Activation Functions

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$ — Smooth, outputs 0-1
- **Tanh**: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ — Outputs -1 to 1
- **ReLU**: $\max(0, z)$ — Simple, efficient, most popular today
- **Why needed?** Without them, stacking layers is equivalent to one linear layer!

26

The XOR Problem

- A single perceptron can only learn **linearly separable** patterns.
- **XOR** (exclusive or) is NOT linearly separable:
- $(0, 0) \to 0$, $(0, 1) \to 1$, $(1, 0) \to 1$, $(1, 1) \to 0$
- **Solution**: Stack multiple layers of neurons!

27

Part 6: Deep Neural Networks

28

Multi-Layer Perceptron (MLP)

- Stack neurons in **layers**: Input → Hidden → Output
- Each layer transforms the representation.
- **Deep** = Many hidden layers.
- Deep networks can learn hierarchical features automatically.

🚀 **Interactive Demo:** neural_network_demo.html

29

Forward Pass: Python Example

PYTHON

```python
def forward(x, W1, b1, W2, b2):
    # Layer 1
    z1 = x @ W1 + b1
    a1 = relu(z1)

    # Layer 2 (output)
    z2 = a1 @ W2 + b2
    output = softmax(z2)

    return output
```

30

Universal Approximation Theorem

- A neural network with **one hidden layer** and enough neurons can approximate **any continuous function** to arbitrary precision.
- This is why neural networks are so powerful!
- In practice, **deeper** networks work better with fewer total neurons.

31

Training: The Big Picture

- **Forward Pass**: Compute predictions from inputs.
- **Loss Calculation**: Measure how wrong we are.
- **Backward Pass**: Compute gradients using chain rule.
- **Update**: Adjust weights to reduce loss.
- **Repeat** for many iterations (epochs).

32

Computational Graphs

- Represent computations as a **directed graph**.
- **Nodes**: Operations (add, multiply, activation).
- **Edges**: Data flow (tensors).
- Enables **automatic differentiation** — compute gradients automatically!

33

The Chain Rule

- For composite functions: $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x}$
- **Backpropagation**: Apply chain rule backwards through the graph.
- Efficiently computes all gradients in one backward pass.
- This is how neural networks learn!

34

Part 7: Gradient Descent & Optimization

35

Why Gradient Descent?

- Closed-form doesn't exist for neural networks.
- Even if it did, matrix inversion is too expensive.
- **Gradient Descent**: Iteratively move towards the minimum.

🚀 **Interactive Demo:** gradient_descent_demo.html

36

Gradient Intuition

- **Gradient**: Vector pointing in direction of steepest **increase**.
- **Negative Gradient**: Points toward steepest **decrease** (what we want!).
- $$\theta_{new} = \theta_{old} - \alpha \cdot \nabla L(\theta)$$
- **Learning Rate** α: Controls step size.

37

Stochastic Gradient Descent (SGD)

- **Batch GD**: Use all m samples → accurate but expensive
- **Stochastic GD**: Use 1 random sample → noisy but fast
- **Mini-batch GD**: Use a batch of 32-256 samples → best of both!
- **Update rule**: $\theta \leftarrow \theta - \alpha \cdot \nabla L(\theta)$
- Most deep learning uses mini-batch SGD.

38

Problems with Vanilla SGD

- **Noisy updates**: Gradient estimate has high variance from sampling.
- **Learning rate sensitivity**: Too high → diverge, too low → very slow.
- **Saddle points**: Gradient is zero but not at minimum.
- **Ravines**: Oscillates across narrow valleys, slow progress along trough.

39

SGD with Momentum

- Add a **velocity** that accumulates past gradients (like a rolling ball):
- **Velocity update**: $v_t = \beta \cdot v_{t-1} + \nabla L(\theta)$
- **Parameter update**: $\theta \leftarrow \theta - \alpha \cdot v_t$
- β is typically 0.9 (momentum coefficient)
- **Intuition**: Builds speed in consistent directions, dampens oscillations.

40

AdaGrad: Adaptive Learning Rates

- **Key idea**: Give each parameter its own learning rate!
- Track sum of squared gradients: $G_t = G_{t-1} + (\nabla L)^2$
- **Update**: $\theta \leftarrow \theta - \frac{\alpha}{\sqrt{G_t + \epsilon}} \cdot \nabla L$
- Parameters with large gradients → smaller steps
- **Problem**: G_t always grows → learning rate shrinks to zero over time.

41

RMSProp: Fixing AdaGrad

- Use **exponential moving average** instead of sum:
- $s_t = \rho \cdot s_{t-1} + (1-\rho) \cdot (\nabla L)^2$
- **Update**: $\theta \leftarrow \theta - \frac{\alpha}{\sqrt{s_t+\epsilon}} \cdot \nabla L$
- $\rho = 0.9$ means we 'forget' old gradients gradually
- **Key insight**: Learning rate adapts but doesn't decay to zero!

42

Adam: The Best of Both Worlds

- Combines **Momentum** + **RMSProp** + **Bias Correction**:
- **Momentum term**: $m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla L$
- **RMSProp term**: $v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla L)^2$
- **Update**: $\theta \leftarrow \theta - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$
- $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected versions (important early in training)

43

Adam: Default Hyperparameters

- **Bias correction**: $\hat{m}_t = \frac{m_t}{1-\beta_1^t}$, $\hat{v}_t = \frac{v_t}{1-\beta_2^t}$
- Needed because m and v start at 0 (biased toward 0 early on)
- **Typical defaults** (work well in most cases):
- $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
- **General advice**: Start with Adam. It's the default choice for most tasks.

44

Summary & Next Steps

45

Key Takeaways

- **ML Paradigms**: Supervised, Unsupervised, Reinforcement Learning
- **Linear Regression**: Has a closed-form solution, but doesn't scale.
- **Neural Networks**: Layers of neurons that learn hierarchical features.
- **Backpropagation**: Chain rule enables efficient gradient computation.
- **Optimizers**: Adam combines momentum and adaptive learning rates.

46

Next Lecture Preview

- **PyTorch**: The framework that handles all this math for us!
- Tensors, Autograd, nn.Module
- Building and training a neural network from scratch

47